WISE ROUND 3: SYSTEM DESIGN — SPEND SQUAD FOCUSED

60 min | Two Engineers | Zoom

Card Product: ordering, transacting, 3DS, bill split, recurring, disputes

---

HOW THIS ROUND WORKS (Reminder)

From the candidate pack:

> "Hypothetically existing feature in a mobile app — identify how it's built, then proposed improvements."

Your JD is Spend Squad. So the feature will be card-related, NOT transfer-related. The interviewers will describe a card feature and ask you to reverse-engineer the mobile architecture.

The format:

1. They describe a feature (5 min)
2. You ask clarifying questions (2 min)
3. You explain the architecture (15-20 min)
4. They propose improvements (10-15 min)
5. You extend the design (10-15 min)
6. Discussion: testing, production, trade-offs (5 min)

THE 6 MOST LIKELY SCENARIOS (from your JD)

SCENARIO 1: CARD TRANSACTION & REAL-TIME NOTIFICATION

Probability: ★★★★★ (Most likely — core product)

Their prompt:

> "When a customer pays with their Wise card, they get an instant notification showing the amount, merchant, and currency. How would you architect this on mobile?"

Clarifying Questions:

> "How quickly should the notification appear after the card tap — under 1 second?"
> "Should we show the notification differently if the app is in foreground vs background?"
> "Is the transaction amount always final, or can it change (like at restaurants with tips)?"
> "Should we handle multi-currency — card charged in EUR but balance in SGD?"

Architecture:

```
FLOW:
Card tap at POS → Visa/Mastercard → Wise Payment Processor
→ Wise Backend → Push Notification (APNs) → Mobile App

┌──────────────────────────────────────────────────────────────────────┐
│                BACKEND                                               │
│                                                                      │
│  Card Network → Payment Processor → Event Bus                        │
│                                  (Kafka)                             │
│                                    │          │                      │
│                  ┌─────────────────────────────┘                     │
│                  ▼                ▼                                  │
│            Notification       Transaction                            │
│            Service            Service                                │
│            (APNs push)        (Store txn)                            │
└──────────────────────────────┬──────────────────────────────────────┘
                               │              │
┌──────────────────────────────┼──────────────┼──────────────────────┐
│            MOBILE APP                                                │
│                                                                      │
│  Push Handler ──────────────────────────────────────┐                │
│      │                        │        │                            │
│      ▼                        ▼        │                            │
│  ┌──────────────────┐   ┌──────────────────────────────────┐        │
│  │ Foreground:      │   │ Background:                      │        │
│  │ In-app banner    │   │ System notification              │        │
│  │ with haptic      │   │ Rich notification with           │        │
│  │                  │   │ merchant logo                    │        │
│  └──────────┬───────┘   └──────────┬───────────────────────┘        │
│      │              │              │                                 │
│      ▼              ▼              │                                 │
│  ┌──────────────────────────────────────────────────────────────┐   │
│  │       TransactionListViewModel                               │   │
│  │  @Published transactions: [Transaction]                      │   │
│  │                                                              │   │
│  │  1. Receive push → parse transaction data                    │   │
│  │  2. Insert at top of local list (optimistic)                 │   │
│  │  3. Refresh from API to confirm                              │   │
│  │  4. Update balance (deduct amount)                           │   │
│  └──────────────────────────────────────────────────────────────┘   │
│                 │                    │                               │
│  ┌──────────────────────────────┼──────────────────────────────┐   │
│  │       TransactionListView                                    │   │
│  │  • Grouped by date (Today, Yesterday, etc.)                  │   │
```

```
│  │ • Each row: merchant icon, name, amount        │ │
│  │ • Pull-to-refresh                              │ │
│  │ • Tap → TransactionDetailView                  │ │
│  └──────────────────────────────────────────────┘ │
└──────────────────────────────────────────────────┘
```

Mobile Code Structure:

```swift
```

```swift
// Transaction model
struct CardTransaction: Codable, Identifiable {
    let id: String
    let merchantName: String
    let merchantCategory: String
    let amount: Decimal
    let currency: String
    let balanceCurrency: String
    let convertedAmount: Decimal?  // If different currency
    let rate: Decimal?
    let status: TransactionStatus
    let timestamp: Date
}

enum TransactionStatus: String, Codable {
    case pending      // Authorization hold
    case completed    // Settled
    case declined     // Rejected
    case reversed     // Refunded
}

// Push notification handling
class NotificationHandler {
    func handle(_ notification: UNNotification) {
        let data = notification.request.content.userInfo

        guard let txnData = data["transaction"] as? [String: Any],
              let txn = parseTransaction(txnData) else { return }

        // Optimistic insert — show immediately
        TransactionStore.shared.insert(txn)

        // Refresh balance
        Task { await BalanceService.shared.refresh() }
    }
}

// Rich notification (UNNotificationServiceExtension)
// Download merchant logo, format amount with currency symbol
// "💳 Starbucks — SGD 6.50 from your SGD balance"
```

Multi-Currency Handling:

```swift
```

```swift
// Card charged in EUR but balance in SGD:
// Show BOTH amounts to user

struct MultiCurrencyTransaction {
    let chargedAmount: Decimal      // €45.00 (what merchant charged)
    let chargedCurrency: String     // EUR
    let deductedAmount: Decimal     // S$65.25 (what left your balance)
    let deductedCurrency: String    // SGD
    let rate: Decimal               // 1.45 (EUR → SGD)

    // Display: "€45.00 at Starbucks Paris"
    //          "S$65.25 from your SGD balance"
    //          "Rate: 1 EUR = 1.45 SGD"
}
```

What to SAY about trade-offs:

> "For the notification, I'd use APNs push rather than polling because card transactions need sub-second notification. Polling every second would drain the battery. Push is event-driven — the server sends it only when something happens.
> For the transaction list update, I'd do an optimistic insert from the push data, then confirm with an API refresh in the background. This means the user sees the transaction immediately, and if there's any discrepancy (like the push had preliminary data), it gets corrected within seconds."

Their improvement — "Now add spending categories and monthly summary":

```swift
```

```
// Backend assigns category per transaction
// Mobile displays chart + filter

class SpendingSummaryViewModel: ObservableObject {
    @Published var categories: [SpendingCategory] = []
    @Published var totalSpend: Decimal = 0
    @Published var monthOverMonthChange: Decimal = 0

    struct SpendingCategory: Identifiable {
        let id: String
        let name: String          // "Food & Drink"
        let total: Decimal        // S$342.50
        let percentage: Double    // 28%
        let transactionCount: Int // 15 transactions
    }

    // Fetch: GET /v3/spend/profiles/{id}/summary?month=2026-02
    // Display: Donut chart using Swift Charts (iOS 16+)
    // Tap category → filtered transaction list
}
```

SCENARIO 2: 3DS AUTHENTICATION

Probability: ★★★★ (In your JD + Wise has public 3DS API)

Their prompt:

> "When a customer makes an online purchase, sometimes 3D Secure authentication is required. How would you architect the mobile 3DS flow?"

How 3DS Actually Works at Wise (from their API docs):

```
FLOW:
1. Customer uses Wise card online
2. Merchant requests 3DS authentication
3. Wise backend receives challenge
4. Wise sends push notification to customer's mobile app
5. Customer sees: "Approve S$150 at Amazon?" → Approve / Decline
6. App sends result to Wise backend
7. Wise tells card network → transaction approved/declined

TIMELINE:
  Card used → Push notification → User taps → Result sent
```

```
│                                │
└──────── Must happen within 5 minutes ────────────────┘
```

Mobile Architecture:

```swift
```

```swift
// 3DS Challenge — received via push notification
struct ThreeDSChallenge: Codable {
    let transactionReference: String
    let merchantName: String
    let amount: Decimal
    let currency: String
    let expiresAt: Date
}

// ViewModel
class ThreeDSViewModel: ObservableObject {
    @Published var challenge: ThreeDSChallenge?
    @Published var timeRemaining: Int = 300  // 5 min countdown
    @Published var isSubmitting = false

    private let apiClient: SpendAPIClient
    private var timer: Timer?

    func startCountdown() {
        timer = Timer.scheduledTimer(withTimeInterval: 1, repeats: true) { [weak self] _ in
            guard let self = self else { return }
            self.timeRemaining -= 1
            if self.timeRemaining <= 0 {
                self.timer?.invalidate()
                self.challenge = nil  // Expired
            }
        }
    }

    func approve() async {
        await submitResult(status: .approved)
    }

    func decline() async {
        await submitResult(status: .declined)
    }

    private func submitResult(status: ChallengeStatus) async {
        isSubmitting = true
        defer { isSubmitting = false }

        guard let challenge = challenge else { return }

        // POST /v3/spend/profiles/{id}/3dsecure/challenge-result
        try? await apiClient.submit3DSResult(
            transactionReference: challenge.transactionReference,
```

```
            status: status
        )
    }

    enum ChallengeStatus: String {
        case approved = "APPROVED"
        case declined = "DECLINED"
    }
}

// View — shows as a modal/sheet over current screen
struct ThreeDSChallengeView: View {
    @ObservedObject var vm: ThreeDSViewModel

    var body: some View {
        VStack(spacing: 20) {
            // Merchant + amount
            Text("Approve payment?")
                .font(.title2)
            Text("\(vm.challenge?.currency ?? "") \(vm.challenge?.amount ?? 0)")
                .font(.largeTitle).bold()
            Text("at \(vm.challenge?.merchantName ?? "")")

            // Countdown
            Text("Expires in \(vm.timeRemaining)s")
                .foregroundColor(vm.timeRemaining < 60 ? .red : .secondary)

            // Buttons
            HStack(spacing: 20) {
                Button("Decline") { Task { await vm.decline() } }
                    .foregroundColor(.red)
                Button("Approve") { Task { await vm.approve() } }
                    .buttonStyle(.borderedProminent)
            }
            .disabled(vm.isSubmitting)
        }
    }
}
```

Key Concerns to Mention:

> **Timing:** "The 3DS challenge has a 5-minute expiry. The push must arrive within seconds, and the UI must make it obvious how much time is left. I'd add a countdown timer and change the color to red when under 60 seconds."
> **Deep linking:** "The push notification must deep-link directly to the 3DS approval screen, even if the app was closed. I'd handle this in the notification's userInfo payload with a custom URL scheme or

> UNNotificationAction."
> **Security:** "The approval should require biometric authentication (Face ID/Touch ID) before submitting — you don't want someone else approving a transaction on a stolen phone."
> **Idempotency:** "Wise's API says 'only the first call will be processed, duplicates ignored.' So if the user taps Approve twice, or if the network retries, it's safe."
> **Offline:** "If the phone is offline when the push arrives, APNs queues it. But if the user doesn't come online within 5 minutes, the challenge expires. We should show 'Challenge expired' if they open the notification too late."

SCENARIO 3: CARD ORDERING FLOW

Probability: ★★★★ (In your JD)

Their prompt:

> "Walk us through how you'd architect the card ordering experience on mobile."

Architecture:

```
USER JOURNEY:
Choose card type → Enter delivery address → Review → Confirm → Track delivery

COORDINATOR PATTERN (multi-step flow):

CardOrderCoordinator
├── Step 1: CardTypeSelectionView
│   └── Physical card / Virtual card
│   └── Debit / Prepaid
│
├── Step 2: DeliveryAddressView (physical only)
│   └── Saved addresses or enter new
│   └── Address validation API
│
├── Step 3: ReviewOrderView
│   └── Card type, delivery address, estimated delivery
│   └── Any fees displayed
│
├── Step 4: ConfirmationView
│   └── Biometric auth before confirming
│   └── POST /v3/spend/profiles/{id}/cards/order
│   └── Idempotency key in header
│
└── Step 5: TrackingView
    └── Card status: Ordered → Printed → Shipped → Delivered
```

```
└── Push notifications at each stage
└── Virtual card: instantly available in Apple Wallet
```

Key Discussion Points:

> **Virtual vs Physical:** "Virtual cards can be issued instantly — no shipping. I'd show the virtual card immediately in-app with the card number, and offer 'Add to Apple Wallet' via PassKit. Physical cards need tracking and a multi-day wait."
> **Dynamic Forms:** "I know Wise uses Dynamic Forms — the backend sends the form structure. For card ordering, this means different countries might require different fields (like ID verification in some countries). The mobile renders whatever the backend sends."

SCENARIO 4: BILL SPLIT

Probability: ★★★ (In your JD)

Their prompt:

> "We want to add bill splitting to the Wise card. After a restaurant payment, the user can split the bill with friends. How would you design this?"

Architecture:

```
FLOW:
User pays → sees transaction → taps "Split" → selects friends
→ enters amounts → sends requests → tracks who paid

┌──────────────────────────────────────────────────────────────┐
│ SplitBillViewModel                                           │
│                                                              │
│ @Published transaction: CardTransaction                      │
│ @Published participants: [Participant]                       │
│ @Published splitType: SplitType                              │
│                                                              │
│ enum SplitType {                                             │
│     case equal          // Divide equally                    │
│     case custom(amounts) // Each person's share              │
│     case percentage     // By percentage                     │
│ }                                                            │
│                                                              │
│ struct Participant {                                         │
│     let contact: Contact   // Wise user or not?              │
│     var amount: Decimal                                      │
│     var status: PaymentStatus                                │
│     // .pending, .paid, .declined                            │
│ }                                                            │
```

```
│                                        │
│  func splitEqually() {                 │
│     let share = transaction.amount / count      │
│     // Handle rounding — last person gets remainder │
│  }                                     │
│                                        │
│  func sendRequests() async {           │
│     for participant in participants {  │
│        if participant.isWiseUser {     │
│           // Wise-to-Wise request (free, instant) │
│        } else {                        │
│           // Send link via SMS/WhatsApp        │
│        }                               │
│     }                                  │
│  }                                     │
└────────────────────────────────────────┘
```

Key Discussion Points:

> **Rounding:** "S$100 split 3 ways is S$33.33, S$33.33, S$33.34. The last person absorbs the rounding difference. This is a fintech detail — we can't lose or create money through rounding."
> **Wise-to-Wise vs external:** "If the friend has Wise, it's a free instant request. If not, we send a payment link. Two very different flows."
> **Notifications:** "Push notifications when someone pays their share. Show a progress bar: '2 of 4 friends have paid.'"

SCENARIO 5: RECURRING PAYMENTS / SUBSCRIPTIONS

Probability: ★★★ (In your JD)

Their prompt:

> "Users want to see and manage their recurring card payments (subscriptions like Netflix, Spotify). How would you design this?"

Architecture:

```swift
```

```swift
// Backend detects recurring patterns from transaction history
// Mobile displays and lets user manage them

struct RecurringPayment: Identifiable, Codable {
    let id: String
    let merchantName: String
    let merchantLogo: URL?
    let amount: Decimal
    let currency: String
    let frequency: Frequency        // weekly, monthly, yearly
    let nextExpectedDate: Date
    let lastChargeDate: Date
    let isActive: Bool

    enum Frequency: String, Codable {
        case weekly, monthly, quarterly, yearly
    }
}

class RecurringPaymentsViewModel: ObservableObject {
    @Published var subscriptions: [RecurringPayment] = []
    @Published var totalMonthly: Decimal = 0

    // "You spend S$89.97/month on subscriptions"
    // Netflix S$16.98, Spotify S$9.99, Gym S$63.00

    func blockMerchant(_ merchantId: String) async {
        // Add to spend controls — block future charges
        // POST /v3/spend/profiles/{id}/cards/{cardId}/controls
    }
}
```

Key Discussion:

> "The tricky part is detection — the backend identifies recurring patterns by analyzing transaction history for the same merchant at regular intervals with similar amounts. The mobile just displays what the backend has identified.
> For management, users should be able to: see all recurring payments in one place, get notified before each charge, block a merchant to prevent future charges (using Wise's spend controls API), and see total monthly subscription spend."

SCENARIO 6: DISPUTE / CHARGEBACK

Probability: ★★★ (In your JD)

Their prompt:

> "A customer doesn't recognize a card transaction and wants to dispute it. How would you design the dispute flow?"

Architecture:

```
FLOW:
User sees unknown transaction → taps "Something wrong?"
→ selects reason → provides details → submits dispute → tracks resolution

DisputeFlowCoordinator
├── Step 1: SelectReasonView
│   ├── "I don't recognize this transaction"
│   ├── "I was charged the wrong amount"
│   ├── "I was charged twice"
│   ├── "I returned the item but wasn't refunded"
│   └── "The item/service was not received"
│
├── Step 2: ProvideDetailsView
│   ├── Text description
│   ├── Upload evidence (photos, receipts)
│   └── Expected resolution amount
│
├── Step 3: ReviewDisputeView
│   ├── Summary of claim
│   └── "Provisional credit may be applied within 48 hours"
│
└── Step 4: DisputeTrackingView
    ├── Status: Submitted → Under Review → Resolved
    ├── Timeline with updates
    └── Push notifications on status changes
```

Key Discussion:

> "Disputes have regulatory timelines — card networks give customers 120 days to dispute, and issuers have specific response windows. The mobile app needs to show these deadlines clearly.
> For evidence upload, I'd compress images before uploading and use multi-part form data with progress indication. Large files should be resumable in case of network interruption.
> Provisional credit is interesting from a mobile perspective — the balance might temporarily increase, then decrease again if the dispute is denied. The UI needs to show provisional vs settled amounts clearly."

CROSS-CUTTING CONCERNS FOR ANY SCENARIO

Always mention these:

Spend Controls (Unique to Card Product)

```swift
// Users can set limits on their card
struct SpendControls {
    var contactlessLimit: Decimal?      // Max per contactless payment
    var onlinePaymentsEnabled: Bool     // Enable/disable online
    var atmWithdrawalsEnabled: Bool     // Enable/disable ATM
    var magneticStripeEnabled: Bool     // Enable/disable swipe
    var monthlySpendLimit: Decimal?     // Total monthly cap
}
// The mobile app checks these controls and shows appropriate
// messages when transactions are declined
```

Card Freeze/Unfreeze

> "Users can freeze their card instantly from the app. This should be a single tap with immediate effect — the backend must process this within milliseconds because the next card transaction could come any second."

Apple Wallet Integration

> "For virtual cards, I'd use PassKit to add the card to Apple Wallet. For physical cards, Apple Wallet shows the card for tap-to-pay via NFC."

YOUR BFX ADVANTAGE — SPEND CONTEXT

Spend Feature	Your PayPal Experience
Card authorization	BFX — currency conversion at point of sale for Visa cards
3DS authentication	PayPal checkout — authentication flows
Multi-currency	BFX — converting across currencies for multi-currency cards
Transaction notifications	RUNE — real-time display of payment status
Spend controls	Express Checkout — enabling/disabling payment methods by country
Dispute resolution	PayPal buyer protection — dispute flows

What to SAY:

> "The Spend product maps directly to my BFX work at PayPal. BFX handles what happens when a Visa multi-currency card is used for a purchase — the authorization, currency conversion decision, and fee calculation. I've worked on the exact backend systems that Wise's card product relies on. The difference is Wise is the issuer, while PayPal is the processor — but the authorization flow and multi-currency logic are the same."

HOW TO APPROACH ANY SCENARIO THEY GIVE

```
STEP 1: Clarify (2 min)
  "How quickly should..."
  "Should this work offline..."
  "Is this iOS only..."

STEP 2: Draw the flow (3 min)
  User action → Mobile → Backend → External service → Back

STEP 3: Mobile architecture (10 min)
  ViewModel + View + Repository
  State management: @Published, loading/error/success

STEP 4: Deep dive on the hard part (10 min)
  What makes THIS feature complex?
  - 3DS: timing, deep linking, biometric auth
  - Transactions: real-time notifications, multi-currency
  - Bill split: rounding, Wise vs non-Wise users
  - Disputes: file upload, regulatory timelines

STEP 5: Discuss (5 min)
  Testing, production concerns, feature flags, analytics
```

QUESTIONS TO ASK THEM

1. "How does the mobile team handle server-driven UI for card features? Do you use Dynamic Forms here too?"
2. "What's the latency target for card transaction notifications — under 1 second?"
3. "How do you handle backward compatibility when the card API evolves?"
4. "What's the biggest mobile technical challenge in the Spend product right now?"